



Birla Institute of Technology & Science, Pilani

Work-Integrated Learning Programmes Division

S1-26_DSE ZG532

Introduction to Data Science

Assignment 1 – PS1

Retail Sales, Customer Demographics, and Revenue Analysis

[Weightage 10%] – Total Marks: 10

Read through this entire document very carefully before you start!

General Instructions

1. For Assignment 1, students must implement the assigned practical data-handling problem using only Python Fundamentals and Data Handling topics covered in DSE ZG532 - Introduction to Data Science.
2. The problem statement and dataset in this document are fixed. Students must not replace the assigned dataset, use another version of the dataset, combine it with another dataset, or redefine the stated practical questions.
3. The assigned dataset is Retail Sales Dataset (retail_sales_dataset.csv), Kaggle version 1, hosted on Kaggle and published there by Mohammad Talib. It contains synthetic retail transactions with transaction dates, customer demographics, product categories, quantities, unit prices, and transaction values.
4. The required data file must contain exactly 1,000 records and 9 columns before cleaning. The required columns are: Transaction ID, Date, Customer ID, Gender, Age, Product Category, Quantity, Price per Unit, and Total Amount. Students must stop execution with a clear validation message if the downloaded file does not match this specification.
5. Students must answer the three practical data questions stated in Task 1. These questions are mandatory and must be answered using numerical or aggregated evidence generated within the submitted notebook.
6. Official Kaggle page: <https://www.kaggle.com/datasets/mohammadtalib786/retail-sales-dataset>. Fixed dataset handle: mohammadtalib786/retail-sales-dataset/versions/1. Required file: retail_sales_dataset.csv. Licence: CC0: Public Domain. Students must cite the Kaggle page, Kaggle publisher, fixed handle, filename, licence, and their own access date.
7. The assignment must be implemented using the Python Standard Library, NumPy, SciPy, and Pandas. The kagglehub package is permitted only to download the assigned Kaggle dataset. It must not be used for analysis, transformation, numerical computation, extraction, or aggregation. Visualization, database, web-scraping, automated-profiling, machine-learning, and other analytical libraries must not be used.

8. Students must install kagglehub in Colab with `!pip install -q kagglehub`; call `kagglehub.dataset_download("mohammadtalib786/retail-sales-dataset/versions/1")`; construct the exact file path `retail_sales_dataset.csv` with `pathlib.Path`; and load that file with `pandas.read_csv`. Do not use a Kaggle API token, an unversioned handle, a manual download, a mirror, or a different CSV file. The loading process must be implemented in the notebook and must not depend on manual uploads, mounted drives, local computer paths, or login-dependent links.
9. The complete work must be submitted as one Google Colab notebook named `StudentID_Name_DSEZG532_Assignment1_PS1.ipynb`. It must execute sequentially from the first cell to the last cell without unresolved errors, missing variables, hidden dependencies, or manual intervention.
10. Students must demonstrate appropriate use of Python variables, data types, operators, conditional statements, loops, collections, and reusable functions where they naturally support loading, metadata validation, cleaning, extraction, or aggregation.
11. At least two user-defined functions are mandatory: one parameterized loading-and-validation function in Task 1 and one parameterized cleaning-and-transformation function in Task 2. Each function must have a docstring, parameters, return value, and a demonstrated call.
12. Students must use NumPy arrays and vectorized operations for numerical computation. Unnecessary element-wise Python loops must not be used for calculations that NumPy can perform directly.
13. Students must use either `scipy.stats.describe` or `scipy.stats.zscore` for an appropriate statistical summary or standardized computation and explain the meaning and suitability of the selected SciPy function.
14. Students must inspect missing values, duplicates, data types, category labels, irrelevant columns, and domain-invalid values. If an issue is absent, report its count as zero. Do not introduce artificial errors, and justify every correction, retention, replacement, or removal decision.
15. Data extraction must be performed on the cleaned DataFrame from Task 2. Across the three specified extraction operations in Task 4, students must demonstrate column selection, `loc`, `iloc`, row slicing, Boolean filtering, query, and sorting.
16. Data aggregation must include the specified multi-key `groupby().agg()` summaries and the specified pivot table. Named output fields, suitable aggregation functions, readable sorting, and brief interpretation are required.
17. Machine learning, classification, prediction, machine-learning-oriented feature engineering, dimensionality reduction, clustering, association rules, anomaly-detection modelling, deep learning, and generative AI are outside the scope and must not be used. Only the explicitly required derived DataFrame columns are permitted. The separate NumPy-derived array required in Task 3, together with temporary Series, Boolean masks, arrays, and summary objects needed to complete the prescribed operations, is permitted; these temporary objects must not be added as extra analytical features or additional derived DataFrame columns.
18. All important outputs must remain visible. The notebook must include original analysis, citations, answers to all three questions, at least five evidence-supported findings, limitations, a conclusion, and the generated file `PS1_retail_category_gender_summary.csv`. The CSV need not be submitted separately, but its filename, Colab path, and first five rows must be displayed.

Problem Statement

Analyze the assigned retail-sales dataset to produce reproducible descriptive evidence about revenue, transaction volume, customer demographics, product categories, calendar months, and unit-price bands. The analysis is operational and descriptive; no visualization or predictive model is permitted.

Task 1: Assigned Dataset, Python Fundamentals, and Data Ingestion

Marks: 2

Load the fixed dataset reproducibly and validate that the correct file and schema have been obtained.

Students must:

- Use only retail_sales_dataset.csv from the fixed Kaggle version-1 handle. Report the dataset title, Kaggle publisher, official Kaggle page, fixed handle, exact filename, CC0: Public Domain licence, access date, retail domain, exact initial shape, and descriptions of all nine columns.
- Implement the prescribed ingestion workflow: install kagglehub in Colab with `!pip install -q kagglehub`; call `kagglehub.dataset_download("mohammadtalib786/retail-sales-dataset/versions/1")`; construct the exact file path retail_sales_dataset.csv with `pathlib.Path`; and load that file with `pandas.read_csv`. Do not use a Kaggle API token, an unversioned handle, a manual download, a mirror, or a different CSV file. Display evidence that the expected file, delimiter where applicable, header, and record count were loaded.
- Store the source metadata, expected shape, required column names, category rules, and processing rules in suitable Python collections such as dictionaries, lists, tuples, or sets.
- Use operators, conditionals, and a loop to validate required columns and metadata. The validation must report missing columns, unexpected columns, and shape mismatch before the workflow proceeds.
- Create a reusable loading-and-validation function with the fixed versioned Kaggle handle, exact required filename, expected initial shape, and required columns as parameters. It must download and load only the assigned file, return the validated DataFrame, and raise or report a clear error when the file, shape, or schema validation fails.
- State the following three assigned practical questions exactly and do not replace them:
Q1: Which product-category and gender combination generated the maximum total revenue, and what were its transaction count, total units, and mean transaction value? If two or more combinations share the maximum total revenue, report and interpret every tied combination.
Q2: How did transaction count, total units, total revenue, and mean transaction value vary by calendar month and customer age group?
Q3: How did transaction count, total units, and total revenue vary across unit-price bands and product categories?
- Display representative records and report shape, column names, data types, non-null counts, memory usage, and structural observations. Confirm whether the data is suitable for Tasks 2-5.

Expected Output:

- Complete assigned-dataset citation, metadata, schema, and the three fixed practical questions.
- Reproducible ingestion code using only the prescribed loading method.
- Python collections and explicit validation results for shape and required columns.
- Reusable loading-and-validation function with a demonstrated successful call.
- Dataset preview, data types, non-null counts, memory information, and suitability inference.

Task 2: Data Cleaning and Transformation Using Pandas

Marks: 2

Prepare a consistent, validated, and analysis-ready DataFrame before numerical analysis.

Students must:

- Perform the following dataset-specific quality assessment: validate Date conversion; missing values; exact duplicates; Transaction ID as a unique integer from 1 through 1000; Customer ID as a unique value in the inclusive sequence CUST001 through CUST1000, where CUST001-CUST999 contain three digits and CUST1000 contains four digits; data types; Gender values {Female, Male}; Product Category values {Beauty, Clothing, Electronics}; surrounding spaces; Age between 18 and 100 inclusive; Quantity as a positive integer; non-negative Price per Unit and Total Amount; and the identity Total Amount = Quantity * Price per Unit within an absolute floating-point tolerance of 0.01. Report zero for an absent issue. Do not drop an identifier until its uniqueness has been verified.
- Produce a before-cleaning quality summary containing row count, column count, total and column-wise missing-value counts, exact-duplicate count, data types, and every specified domain-validity check.
- Apply the following required standardization and type conversions: rename all columns exactly to transaction_id, date, customer_id, gender, age, product_category, quantity, price_per_unit, and total_amount; parse date as datetime; strip surrounding spaces from text columns; standardize gender and product_category to title case; and convert the standardized categorical columns to the category data type.
- Demonstrate at least two appropriate Pandas transformation operations selected from string methods, datetime methods, replace, map, apply, astype, and pandas.cut. Explain why each selected operation is appropriate for this dataset.
- Create the following required derived DataFrame columns exactly as specified: (i) month_period = date.dt.to_period("M"), stored as a monthly Period value displayed in YYYY-MM format; (ii) age_group = pandas.cut(age, bins=[18, 30, 45, 60, 100], labels=["18-30", "31-45", "46-60", "61+"], include_lowest=True, right=True, ordered=True); and (iii) unit_price_band = pandas.cut(price_per_unit, bins=[0, 50, 300, float("inf")], labels=["Low", "Standard", "Premium"], include_lowest=True, right=True, ordered=True). The resulting intervals are 0-50, >50-300, and >300 respectively. Values outside the validated non-negative range must not be silently assigned to a band.
- Create a parameterized cleaning-and-transformation function that returns the cleaned DataFrame. Display a before-and-after comparison of shape, missing values, duplicates, data types, category labels, and domain-validity checks.

Expected Output:

- Before-cleaning data-quality report with issue counts and dataset-specific validity checks.
- Pandas cleaning, standardization, conversion, and transformation code with justifications.
- All explicitly required derived DataFrame columns, formulas or rules, data types, and sample values.
- Reusable cleaning-and-transformation function with a demonstrated call.
- Before-and-after quality comparison and cleaned DataFrame preview.

Task 3: Numerical Data Handling Using NumPy and SciPy

Marks: 2

Perform efficient numerical computations on the cleaned DataFrame using fixed columns and a fixed

derived-array definition.

Students must:

- From the cleaned DataFrame, select the following numerical columns: age, quantity, price_per_unit, and total_amount. Convert or access them as NumPy arrays with appropriate numerical data types.
- Demonstrate array shape, number of dimensions, indexing, slicing, Boolean masking, and at least two vectorized arithmetic operations using the selected numerical data.
- For every selected numerical column, compute minimum, maximum, mean, median, population standard deviation, population variance, and the 25th, 50th, and 75th percentiles. Use ddof=0 for both population standard deviation and population variance, handle NaN values explicitly, and present the results in one labelled summary table.
- Create the following required vectorized NumPy-derived array: `recalculated_amount = quantity_array * price_per_unit_array`. Compare this array element-wise with `total_amount_array` by using `numpy.isclose` with `atol=0.01`, report the number of mismatches, and display representative values. Keep the array as a separate Task 3 output and do not insert it into the cleaned DataFrame or count it as a Task 2 derived column.
- Use `scipy.stats.describe` or `scipy.stats.zscore` on at least one selected numerical array. Display the relevant result, justify the selected function, and explain its practical meaning. If `scipy.stats.describe` is selected, use `ddof=0` so that its variance is consistent with the required population variance; if `zscore` is selected, use `ddof=0` and do not use the result to remove records or perform anomaly detection.
- Interpret the numerical summaries, the required derived array, and the SciPy output using specific values. Do not merely repeat the displayed table.

Expected Output:

- NumPy arrays for every specified numerical column with properties, indexing, slicing, and masking.
- Labelled summary table containing all mandatory statistics for every selected column.
- Required vectorized derived NumPy array with formula, representative values, and interpretation.
- SciPy result with function justification and practical explanation.
- Evidence-based numerical inferences from the cleaned data.

Task 4: Data Extraction and Aggregation

Marks: 2

Execute the fixed extraction and aggregation operations to produce comparable outcomes across submissions.

Students must:

- Use `loc` with column selection and multiple Boolean conditions to retrieve transactions whose `total_amount` is above the cleaned-data median and whose `quantity` is at least 3. Display `date`, `transaction_id`, `gender`, `age_group`, `product_category`, `quantity`, `price_per_unit`, and `total_amount`; sort by `total_amount` and `quantity` in descending order.
- Use `query` to retrieve customers aged 31-45 who purchased Clothing or Electronics. Select `date`, `customer_id`, `gender`, `age`, `product_category`, `quantity`, `price_per_unit`, and `total_amount`; sort by `date` in ascending order and `total_amount` in descending order.

- Create a chronologically sorted subset containing date, transaction_id, customer_id, product_category, quantity, price_per_unit, and total_amount; then use iloc row and column slicing to display the final 10 rows and the first 7 columns. Report the number of records produced by every extraction and interpret each displayed result.
- Create a product_category-gender groupby().agg() summary using observed=True and the following exact named aggregations: transaction_count = size of transaction_id; total_units = sum of quantity; total_revenue = sum of total_amount; mean_transaction_value = mean of total_amount; and median_transaction_value = median of total_amount. Sort by total_revenue in descending order.
- Create a month_period-age_group grouped summary using observed=True with transaction_count = size of transaction_id, total_units = sum of quantity, total_revenue = sum of total_amount, and mean_transaction_value = mean of total_amount. Preserve chronological month order and the defined age-group order.
- Create a unit_price-band-product_category grouped summary using observed=True with transaction_count = size of transaction_id, total_units = sum of quantity, total_revenue = sum of total_amount, and mean_transaction_value = mean of total_amount. Preserve the Low, Standard, Premium price-band order and sort categories within each band by total_revenue in descending order.
- Create a pivot table with age_group as the index, product_category as the columns, total_amount as the values, and sum as the aggregation function. Use observed=False, dropna=False, and fill_value=0; reindex if necessary to preserve the defined age-group order and the product-category column order ["Beauty", "Clothing", "Electronics"]. Display the completed pivot table and interpret its most important comparison.
- Export the first specified groupby summary as PS1_retail_category_gender_summary.csv with index=False after resetting the grouped index. Display the filename, Colab path, file existence check, shape, and first five rows of the exported report.

Expected Output:

- Code, record counts, displayed results, purposes, and interpretations for all three fixed extraction operations.
- All specified multi-key groupby().agg() summaries with named output columns and readable sorting.
- Specified pivot table with correct index, columns, values, aggregation function, and ordering.
- Generated PS1_retail_category_gender_summary.csv, displayed path, existence check, shape, and first-five-row preview.
- Evidence-based interpretation linking the extraction and aggregation results to the fixed questions.

Task 5: Integrated Data Pipeline, Insights, and Conclusion

Marks: 2

Integrate the completed work into a reproducible notebook and communicate evidence-based conclusions.

Students must:

- Organize the notebook in the sequence Input -> Ingestion -> Validation -> Cleaning and Transformation -> Numerical Analysis -> Extraction -> Aggregation -> Output. Task headings and execution order must follow this sequence.
- Call the Task 1 loading-and-validation function and the Task 2 cleaning-and-transformation function in a repeatable pipeline. Clearly identify function inputs, returned objects, and the DataFrame passed to each later stage.

- Verify the final cleaned dataset and outputs by reporting the validated initial shape, the count removed under each cleaning rule, any overlap among removal rules, and the resulting final cleaned shape. Also verify required-column presence, data types, remaining missing-value count, duplicate treatment, specified domain rules, derived-column presence, aggregation-column presence, and exported-file existence.
- Answer Q1, Q2, and Q3 exactly as stated in Task 1 by using specific numerical values from the notebook. For each answer, identify the supporting NumPy, SciPy, extraction, groupby, or pivot-table output.
- Present at least five meaningful findings supported by NumPy, SciPy, extraction, groupby, or pivot-table evidence. Answers to the three fixed questions may be included among the five findings, but every finding must cite its supporting value or table.
- Discuss dataset and workflow limitations, provide a concise conclusion for the assigned professional context, cite the assigned Kaggle dataset and any reused code, and refer explicitly to the exported report PS1_retail_category_gender_summary.csv.

Expected Output:

- Concise Input -> Process -> Output summary of the complete workflow.
- Successfully rerun function-based pipeline with final validation results.
- Direct, numerical answers to all three assigned practical questions.
- At least five evidence-supported findings linked to displayed outputs, followed by final limitations, conclusion, dataset and code references, and reference to PS1_retail_category_gender_summary.csv.